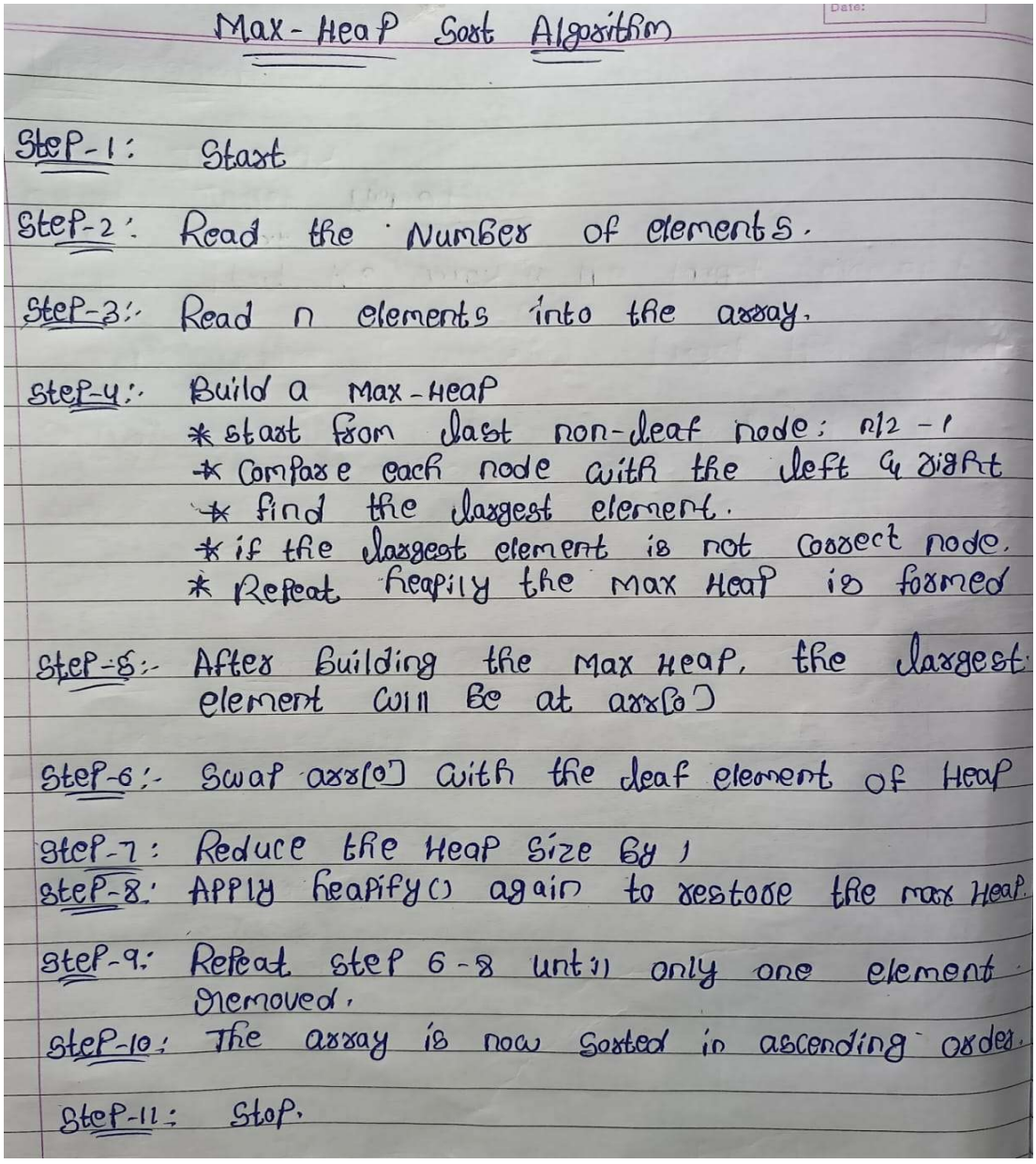
Practical: 3	Implementation of max-heap sort algorithm.
05/08/2026	

Aim: To Implement of max-heap sort algorithms.

MAX-HEAP SORT

Algorithm:



Max-Heap Sort Algorithm

Step-1: Start

Step-2: Read the Number of elements.

Step-3: Read n elements into the array.

Step-4: Build a Max-Heap
* start from last non-leaf node: n/2 - 1
* Compare each node with the left & right
* find the largest element.
* if the largest element is not correct node.
* Repeat heapify the max Heap is formed

Step-5: After building the max Heap, the largest element will be at arr[0]

Step-6: Swap arr[0] with the leaf element of Heap

Step-7: Reduce the Heap Size by 1

Step-8: Apply heapify() again to restore the max Heap.

Step-9: Repeat step 6-8 until only one element removed.

Step-10: The array is now sorted in ascending order.

Step-11: Stop.

Program:

```
#include <iostream>
using namespace std;
void heapify(int arr[], int n, int i) {
   int largest = i;
   int left = 2 * i + 1;
   int right = 2 * i + 2;
   if (left < n && arr[left] > arr[largest]) {
      largest = left;
   }
   if (right < n && arr[right] > arr[largest]) {
      largest = right;
   }
   if (largest != i) {
      swap(arr[i], arr[largest]);
      heapify(arr, n, largest);
   }
}
void heapSort(int arr[], int n) {
   for (int i = n / 2 - 1; i >= 0; i--) {
      heapify(arr, n, i);
   }
   for (int i = n - 1; i > 0; i--) {
      swap(arr[0], arr[i]);
      heapify(arr, i, 0);
   }
}
int main() {
   int arr[] = {4, 10, 3, 5, 1};
   int n = 5;
   heapSort(arr, n);
   for (int i = 0; i < n; i++) {
      cout << arr[i] << " ";
   }
   return 0;
}
```

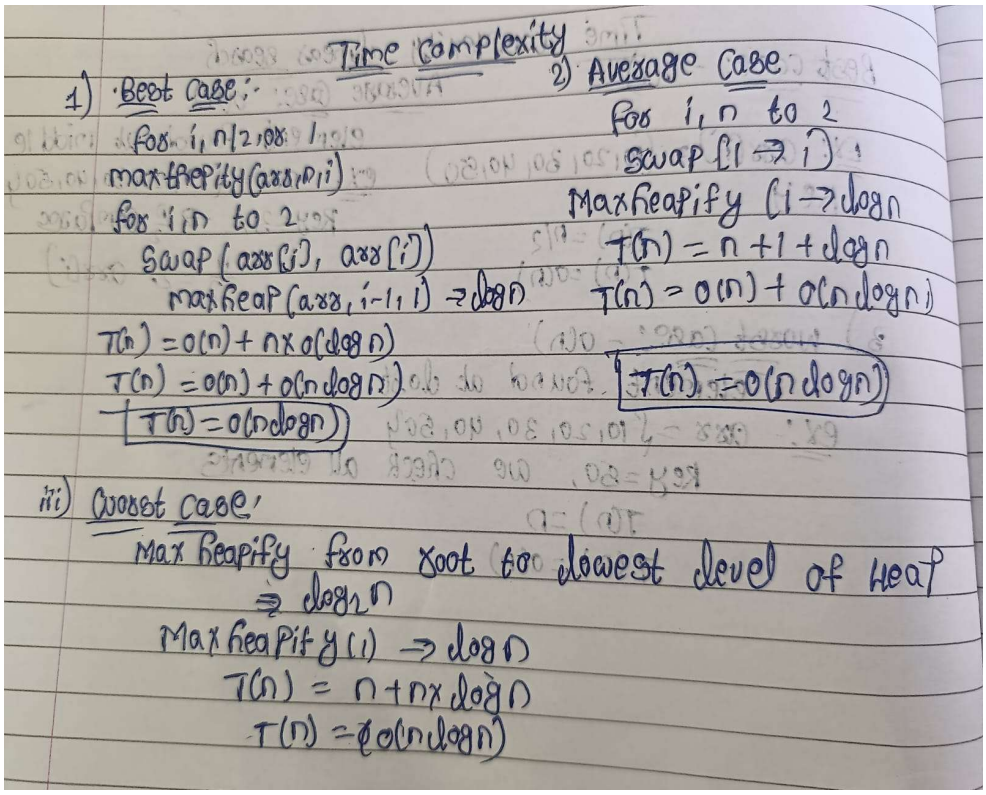
Output:

```
Output

Enter the number of elements: 5
Enter 5 elements:
3
1
6
4
2
Array after Heap Sort: 1 2 3 4 6

=== Code Execution Successful ===
```

Time Complexity:



Conclusion:

Hence, from this practical of Max Heap Sort, the algorithm and program have been executed successfully